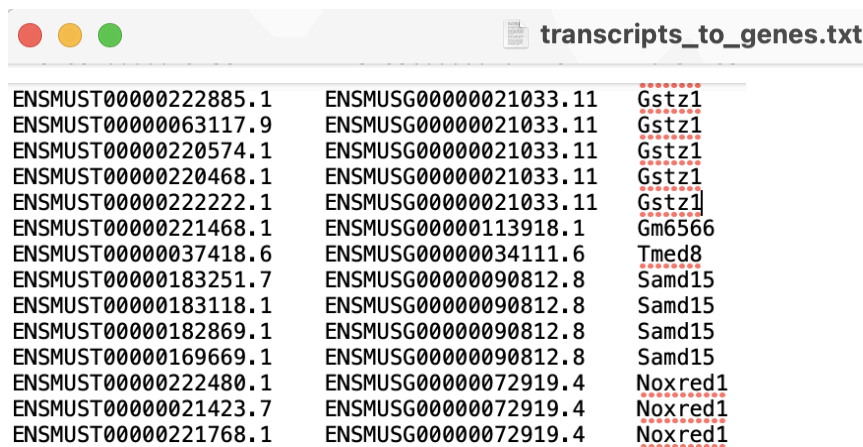


Neuro-genomics - Exercise 5

Python Dictionary: key-value pairs

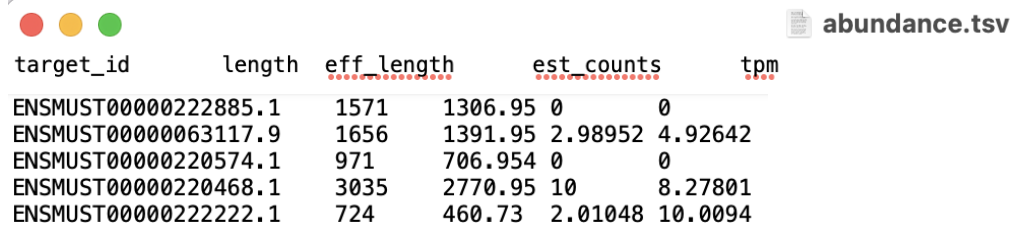
Last week we created a vector of expression using RNAseq data of hippocampal neurons. However, the expression vector was according to transcripts (isoforms), and not according to genes. In this exercise we will map the transcripts into genes, and then we will create a file with the genes sorted according to their expression levels. In the process we will learn about Python Dictionaries, and also practice jupyter and regex some more.

Last week we had two files: one file, 'transcripts_to_genes.txt' was downloaded from kallisto website and contained the link between transcripts (isoforms, first column) and gene names (third column).



Transcript ID	Transcript Name	Gene Name
ENSMUST00000222885.1	ENSMUSG00000021033.11	Gstz1
ENSMUST00000063117.9	ENSMUSG00000021033.11	Gstz1
ENSMUST00000220574.1	ENSMUSG00000021033.11	Gstz1
ENSMUST00000220468.1	ENSMUSG00000021033.11	Gstz1
ENSMUST00000222222.1	ENSMUSG00000021033.11	Gstz1
ENSMUST00000221468.1	ENSMUSG000000113918.1	Gm6566
ENSMUST00000037418.6	ENSMUSG00000034111.6	Tmed8
ENSMUST00000183251.7	ENSMUSG00000090812.8	Samd15
ENSMUST00000183118.1	ENSMUSG00000090812.8	Samd15
ENSMUST00000182869.1	ENSMUSG00000090812.8	Samd15
ENSMUST00000169669.1	ENSMUSG00000090812.8	Samd15
ENSMUST00000222480.1	ENSMUSG00000072919.4	Noxred1
ENSMUST00000021423.7	ENSMUSG00000072919.4	Noxred1
ENSMUST00000221768.1	ENSMUSG00000072919.4	Noxred1

The second file, 'abundance.tsv', was a file we created by running kallisto on the sequencing data.



target_id	length	eff_length	est_counts	tpm
ENSMUST00000222885.1	1571	1306.95	0	0
ENSMUST00000063117.9	1656	1391.95	2.98952	4.92642
ENSMUST00000220574.1	971	706.954	0	0
ENSMUST00000220468.1	3035	2770.95	10	8.27801
ENSMUST00000222222.1	724	460.73	2.01048	10.0094

In this file:

length is the length of the transcript, in bases

eff_length is the effective length of the transcript.

est_counts means estimated counts, marked by X_i . This is the number of reads assigned to each transcript.

tpm which means Transcripts Per Million.

$$\text{TPM}_i = \frac{X_i}{\tilde{l}_i} \cdot \left(\frac{1}{\sum_j \frac{X_j}{\tilde{l}_j}} \right) \cdot 10^6$$

As we discussed last week, counts (or estimated counts) can not be compared within a sample, because every transcript has a different length. Therefore, if we want to quantify a given gene, we need to sum the TPM of the transcripts that are mapped to that gene. For example, for the gene 'Gstz1', you can see the name of the transcripts and their TPM in the fragments of the files above, the summed expression is 4.92642+8.27801+10.0094=23.21383.

In the scripts below we will use [Python Dictionaries](#), which is also known as key-value pair. The idea is simple - in a dictionary each key is associated with a value. The key and the value can be strings, integers, floats or even lists or other dictionaries. The power of dictionaries comes from the ease of creating them and the ease of locating keys or values. Most programming languages have key-value pairs, and a dictionary is called a 'hash' in some languages. It is also present in Matlab and R, but since these languages are based on numbers/vectors/matrices, Matlab/R do not perform well when strings of letters are involved.

Part 1

Read the file 'transcripts_to_genes.txt' and create a dictionary in which the transcript name is a key and the gene name is a value. Using this dictionary we will be able to map each transcript to the associated gene. The jupyter notebook can be found [here](#).

```
# input = the file "transcripts_to_genes.txt"
# output = the dictionary gene_name_of_transcript which holds the gene name for each transcript

import re
file1 = open("/Users/shaharalon/Desktop/Kallisto/mus_musculus/transcripts_to_genes.txt", "r")
gene_name_of_transcript={}
# get the assignment of transcripts to genes
x=file1.readline()
while True:
    # example of a line ENSMUST00000162897.1—ENSMUSG00000051951.5—Xkr4
    match = re.search('^(.+)\t.+\t(.+)\n', x)
    if match:
        transcript_name=match.group(1)
        gene_name=match.group(2)
        gene_name_of_transcript[transcript_name]=gene_name
    else:
        print(x)
    x=file1.readline()
    if not x:
        break
file1.close()
print(gene_name_of_transcript['ENSMUST00000162897.1'])
```

Xkr4

A dictionary is initialized using the curly brackets. In this case, the dictionary is 'gene_name_of_transcript'.

We are using regular expressions to extract the transcript names and the gene names from the file 'transcripts_to_genes.txt'. Since each line in this file contains both the transcript name and the gene name we are using [Group Extraction](#) in regex. Group extraction allows us to get several matches from one string. Basically, to use group extraction one just needs to use round brackets around each pattern which is desired. These patterns are numbered by regex according to the order of the round brackets, and the result will be stored in: match.group(1), match.group(2) and so on (instead of match.group() when no group extraction is used).

To assign keys and values to a dictionary, we are using the square brackets:

```
dictionary[key]=value
```

This is also how we detect the value given a key:

```
dictionary[key]
```

Which will return the value for that specific key

Part 2

The input is the file 'abundance.tsv' and the dictionary 'gene_name_of_transcript'. In this script we create a dictionary in which the gene name is a key and the summed expression, over all the transcripts of a given gene, is the value. Using this dictionary we will be able to know the expression, in TPM, for each gene.

The jupyter notebook below can be found [here](#).

```

# input(1)= the file "abundance.tsv"
# input(2)= the dictionary gene_name_of_transcript
# output = the dictionary tpm_per_gene which holds the summed tpm for each gene

import re
file1 = open("/Users/shaharalon/Desktop/Kallisto/kallisto_results/abundance.tsv", "r")
tpm_per_gene={}
#initialize the tpm_per_gene dictionary
for value in gene_name_of_transcript.values():
    tpm_per_gene[value]=0

# for each gene, sum the tpm of the transcripts that belong to that gene
x=file1.readline() # the first line contains titles
x=file1.readline()
transcript_found=0
transcript_not_found=0
while True:
    # example of a line ENSMUST00000222222.1---724*460.73---2.01048*10.0094
    match = re.search('^(.+)\t.+ \t.+ \t.+ \t(.+)\n', x)
    if match:
        transcript_name=match.group(1)
        tpm=match.group(2)
        # the get method below prevents errors when the transcript_name is not in the dictionary
        gene_name=gene_name_of_transcript.get(transcript_name, 'None') # return None if transcript_name is not found
        if (gene_name != 'None'):
            tpm_per_gene[gene_name]=tpm_per_gene[gene_name]+float(tpm) # we need to cast tpm into float
            transcript_found=transcript_found+1
        else:
            # print('no gene name for this transcript:\t', transcript_name)
            transcript_not_found=transcript_not_found+1
    else:
        print(x)
        x=file1.readline()
        if not x:
            break
file1.close()
print('transcript found:\t', transcript_found)
print('transcript not found:\t', transcript_not_found)
print(tpm_per_gene['Gstz1'])

```

```

transcript found:      116816
transcript not found:  1673
23.21383

```

The script starts with initiation of the new dictionary 'tpm_per_gene', we do that by looping over all the values in the dictionary 'gene_name_of_transcript'. In this loop, each value (i.e. gene name), is a key in the new dictionary. We set the initial value for each key to be zero. Most genes will likely remain with an expression of zero.

Next, we read the abundance file, and use group extraction in regex to collect the transcript name and the TPM is each line. For each transcript name we detect the gene_name using the dictionary 'gene_name_of_transcript'. Note that some transcripts might not be present in the dictionary, therefore we use the method 'get' which doesn't cause error -- gene_name_of_transcript[transcript_name] will return an error if 'transcript_name' is not a key in the dictionary.

Note that we used casting to change the parameter 'tpm'. 'tpm' is a string, because it was created during the regex extraction. We cast it into a number of the type float. Since there is a floating point, casting to integer type would have created an error.

Then, we sum the TPM for each transcript that belongs to a specific gene. To check that the results are ok, we print the gene 'Gstz1', for which we already know the summed TPM for all the transcripts.

Part 3

Finally, in this script, we use the dictionary 'tpm_per_gene' as an input, and return a text file with the summed TPM for all genes, sorted according to decreasing expression. Looking at the top expressed genes, one can learn a lot about the tissue studied. The jupyter notebook can be found [here](#).

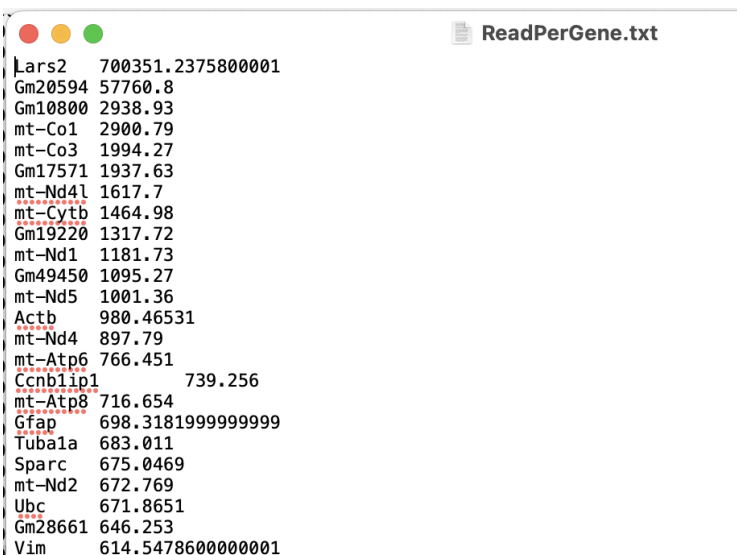
```
# input = the dictionary tpm_per_gene
# output = the file "ReadPerGene.txt"

file1 = open("ReadPerGene.txt", "w")
# sort the dictionary tpm_per_gene by value (summed tpm) and print in descending order to file
sorted_dict = dict(sorted(tpm_per_gene.items(),
                           key=lambda item: item[1],
                           reverse=True)) # sort descending
for key, value in sorted_dict.items():
    x=key+"\t"+str(value)+"\n"; # to concatenate using + we need to cast the float into string
    file1.writelines(x)
file1.close()
```

Here we sorted the dictionary by value. It is also possible to sort a dictionary by key. To learn more about sorting in a dictionary, read [this](#).

Above, we concatenate several strings together using +. We did this to print one resulting string into file. Note that since the value of the 'tpm_per_gene' dictionary is the summed TPM, it is actually a float, and therefore we used casting again. This time we used the command 'str' to cast the number as a string.

Output file



```
ReadPerGene.txt
Lars2 700351.2375800001
Gm20594 57760.8
Gm10800 2938.93
mt-Co1 2900.79
mt-Co3 1994.27
Gm17571 1937.63
mt-Nd4l 1617.7
mt-Cytb 1464.98
Gm19220 1317.72
mt-Nd1 1181.73
Gm49450 1095.27
mt-Nd5 1001.36
Actb 980.46531
mt-Nd4 897.79
mt-Atp6 766.451
Ccnb1ip1 739.256
mt-Atp8 716.654
Gfap 698.3181999999999
Tuba1a 683.011
Sparc 675.0469
mt-Nd2 672.769
Ubc 671.8651
Gm28661 646.253
Vim 614.5478600000001
```

Lars2 is by far the gene with the highest expression. This is interesting (not to say suspicious) given the rule of [Lars2](#). Look at this mouse gene in the UCSC genome browser to see if you can detect why this gene is so highly expressed. Hint: focus on the 3' UTR and open the channel 'RepeatMasker' to display repeat elements, focus on rRNA genes.